Big Data Analytics

Past papers and review questions with answers

MIT 91205 · Days 1 to 4: foundations, Hadoop, Hive and Pig, Spark

github.com/mugishajc/big_data_learning_resources_tools

Day 1: Big Data Foundations and Distributed Systems

Answers to the 15 questions most likely to come up in a CAT or exam on Day 1 (big data concepts, distributed systems, CAP, and reference architectures), plus the Day 1 discussion question on MapReduce failures.

Q1. Define Big Data and explain the 5 V's.

Big Data is data whose size, variety or speed is too much for a single machine or a traditional database to store and analyse within acceptable time and cost. It is not just "a lot of data"; it is data that forces new tools and architectures.

V	Meaning	Example
Volume	Amount of data (terabytes to petabytes)	Years of bank transactions
Velocity	Speed at which data arrives and must be processed	Card payments checked for fraud in milliseconds
Variety	Structured, semi-structured and unstructured types	Tables, JSON logs, images, voice
Veracity	Quality and trustworthiness (noise, gaps, errors)	Duplicate or missing sensor readings
Value	Useful insight gained from the data	Lower stock-outs after demand forecasting

Volume and velocity stress the infrastructure, variety stresses integration, veracity stresses trust, and value is the reason to do the work at all.

Q2. Distinguish structured, semi-structured and unstructured data.

- **Structured:** fixed schema, rows and columns, easy to query with SQL (a relational sales table).
- **Semi-structured:** has tags or keys but no rigid table schema (JSON, XML, server logs).
- **Unstructured:** no predefined model (free text, images, audio, video).

Relational databases are built for structured data. Big Data platforms must handle all three, often in one pipeline, which is why NoSQL stores and schema-on-read tools exist.

Q3. Describe the four types of analytics with an example of each.

Type	Question answered	Example
Descriptive	What happened?	Monthly sales dashboard
Diagnostic	Why did it happen?	Drill-down shows sales fell in one region after a price change
Predictive	What is likely to happen?	Model forecasts next month's demand
Prescriptive	What should we do?	Optimiser recommends stock levels per store

Complexity and business value both rise from descriptive to prescriptive. Each level usually builds on the one below it.

Q4. Explain the Big Data analytics lifecycle.

It has six stages that repeat in a loop:

1. Business understanding: define the problem, goals and success measures.
2. Data acquisition: collect data from sources by batch or streaming ingestion.
3. Data preparation: clean, integrate and transform (often 60 to 80% of the effort).
4. Data analysis: explore, model and evaluate.
5. Visualisation and interpretation: present results to the right audience.
6. Action and feedback: apply the insight, measure the effect, feed lessons back to stage 1.

It is iterative because results from later stages change the questions asked in earlier ones.

Q5. What is a distributed system, and what are its key characteristics?

A distributed system is a group of independent computers that work together over a network and appear to users as one system.
Characteristics:

- Resource sharing (storage, CPU, data across nodes).
- Concurrency (many tasks run at the same time).
- Transparency (users do not see where data or work lives).
- Scalability (add nodes to add capacity).
- Fault tolerance (keeps working when some parts fail).
- Heterogeneity (nodes can differ in hardware and software).

Big Data uses distributed systems because one machine cannot hold or process the data fast enough, and clusters of cheap machines give scale, speed and reliability.

Q6. Compare vertical and horizontal scaling.

	Vertical (scale up)	**Horizontal (scale out)**
How	Bigger machine: more CPU, RAM, disk	More machines in a cluster
Limit	Hardware ceiling of one server	Practically very high
Cost at scale	Expensive	Cheap commodity hardware
Complexity	Low	Needs partitioning, coordination, consistency handling
Failure	Single point of failure	Survives node loss with replication

Big Data platforms (HDFS, Spark, Cassandra) are built for horizontal scaling.

Q7. State the CAP theorem and give examples of CP and AP systems.

The CAP theorem says a distributed data store cannot guarantee all three of these at the same time when a network partition happens:

- **Consistency:** every read sees the latest write.
- **Availability:** every request gets a response.
- **Partition tolerance:** the system keeps working when the network splits.

Partitions cannot be ruled out on a real network, so the practical choice during a partition is between C and A.

- CP (refuse or fail some requests rather than return stale data): HBase, MongoDB with majority write concern.
- AP (always answer, reconcile later): Cassandra, DynamoDB, Riak.
- CA only holds when there is no partition, such as a single-node relational database.

Example: a bank balance service prefers CP; a social media "likes" counter can be AP.

Q8. What does PACELC add to CAP?

CAP only describes behaviour during a partition. PACELC also covers normal operation:

- If there is a Partition (P), choose Availability (A) or Consistency (C).
- Else (E), choose Latency (L) or Consistency (C).

Even with a healthy network, strong consistency needs coordination between replicas, which adds delay. Cassandra is usually described as PA/EL (favours availability and low latency); HBase leans PC/EC.

Q9. Explain the main consistency models.

- Strong consistency: every read returns the most recent write. Simple to reason about, costs latency.
- Eventual consistency: replicas converge if no new writes arrive. Fast and available, reads may be stale for a short time.
- Causal consistency: operations that depend on each other are seen in order by everyone.
- Read-your-writes: a client always sees its own updates.
- Monotonic reads: a client never sees older data after seeing newer data.

Choice depends on the application: payments need strong consistency, a news feed can accept eventual consistency.

Q10. Compare ACID and BASE.

ACID (relational)	BASE (many NoSQL)
Atomicity: all or nothing	Basically Available: system keeps answering under failure
Consistency: valid state to valid state	Soft state: state can change without new input (replicas syncing)
Isolation: transactions do not interfere	Eventual consistency: replicas converge over time
Durability: committed data survives crashes	

ACID favours correctness, BASE favours availability and scale. Many modern systems mix the two (for example, ACID for orders, BASE for product catalogue).

Q11. Describe data partitioning strategies and their main challenge.

Partitioning splits data across nodes so work runs in parallel.

- Range partitioning: by value ranges (dates, ID ranges). Good for range queries, risk of hot spots on recent ranges.
- Hash partitioning: hash of a key decides the node. Even spread, but range queries must touch all nodes.
- List partitioning: explicit values map to partitions (region = East goes to node 2).
- Round-robin: rows dealt out in turn. Even load, no locality by key.

The main challenge is skew: when some keys hold far more data than others, one node becomes a bottleneck. Rebalancing as data grows is the second challenge.

Q12. Describe replication strategies.

Replication keeps copies of data on several nodes for fault tolerance, read throughput and lower latency.

- Master-slave (leader-follower): writes go to one master, reads can go to replicas. Simple, but the master is a bottleneck and a failover point.
- Multi-master: several nodes accept writes. Higher availability, but conflicts must be resolved.
- Quorum-based: a write succeeds after W replicas confirm, a read asks R replicas. If $W + R > N$ (number of replicas), a read overlaps the latest write. Example: $N = 3$, $W = 2$, $R = 2$.

Key design choices are replication factor, where copies are placed, and the consistency level.

Q13. Name the types of failure in a distributed system and the mechanisms used to tolerate them.

Failure types:

- Crash: a node stops.
- Omission: messages or operations are lost.
- Timing: replies arrive too late.
- Byzantine: a node behaves arbitrarily or maliciously.

Fault tolerance mechanisms:

- Replication: extra copies survive a lost node.
- Heartbeats: regular "I am alive" messages detect failed nodes.
- Checkpointing: saved state lets work restart from a known point.
- Retries and speculative execution: rerun failed or slow tasks elsewhere.

Q14. Compare batch, stream and interactive processing.

Model	Data	Goal	Examples
Batch	Bounded, stored dataset	High throughput, minutes to hours	MapReduce, Spark batch jobs
Stream	Unbounded, arriving continuously	Low latency, milliseconds to seconds	Kafka with Spark Structured Streaming
Interactive	Stored data, ad hoc questions	Answers in seconds	Hive, Spark SQL, Presto/Trino

Choose by whether the data is bounded, how fresh the answer must be, and the type of task. Many systems combine models.

Q15. Describe YARN, the layered reference architecture, and Lambda vs Kappa.

YARN manages cluster resources:

- ResourceManager: allocates resources for the whole cluster.
- NodeManager: one per node, launches and monitors containers.
- ApplicationMaster: one per application, requests containers and coordinates its tasks. It lets several frameworks (MapReduce, Spark, Tez) share one cluster.

Layered reference architecture: sources, ingestion, storage, processing, analytics and modelling, visualisation and consumption, with governance and operations (security, metadata, lineage, monitoring) running across all layers.

	Lambda	Kappa
Paths	Batch layer + speed layer + serving layer	One streaming path
Reprocessing	Rerun batch layer	Replay the stored stream (e.g. Kafka)
Strength	Accurate history plus fresh results	One code base, simpler
Weakness	Two code bases to keep in sync	Needs durable, replayable streams; costly for huge history

Q16 (discussion). How does MapReduce handle node failures?

- The master detects failed workers through missing heartbeats.
- Map tasks on a failed node are rerun on another node, even if they had finished, because their output lived on that node's local disk.
- Reduce tasks that had finished are not rerun, because their output is already in the distributed file system (GFS/HDFS).
- Input is replicated (three copies by default), so the rerun task reads another replica, ideally a local one.
- Slow tasks near the end of a job get backup copies (speculative execution); the first to finish wins.

These are Day 1 mechanisms in practice: heartbeats, replication, re-execution and speculation. Tasks must be deterministic so a rerun gives the same result.

Quick recall

- 5 V's: volume, velocity, variety, veracity, value.
- Analytics types in order: descriptive, diagnostic, predictive, prescriptive.
- Data preparation is usually 60 to 80% of project effort.

- Horizontal scaling (more machines) is the basis of Big Data platforms.
- CAP: during a partition, pick consistency or availability.
- CP examples: HBase, MongoDB (majority writes). AP examples: Cassandra, DynamoDB.
- PACELC: else (no partition), trade latency against consistency.
- ACID favours correctness; BASE favours availability and scale.
- Quorum rule: $W + R > N$ gives read-after-write overlap.
- Hash partitioning spreads load; range partitioning helps range queries; skew hurts both.
- YARN: ResourceManager, NodeManager, ApplicationMaster.
- Lambda has two processing paths; Kappa has one replayable stream.

Day 2: HDFS, YARN and MapReduce

Answers to the 15 Day 2 Knowledge Review questions, plus a worked storage calculation and a step-by-step Word Count trace.

Q1. Why does HDFS separate metadata from data storage? What would happen if all metadata were stored on DataNodes?

The NameNode holds only metadata (file names, directory tree, block list, block locations) in memory, while DataNodes hold the actual blocks on disk. This keeps lookups fast, gives one authoritative view of the namespace, and lets bulk data flow directly between clients and DataNodes, so the NameNode never becomes a data-transfer bottleneck.
If metadata lived on DataNodes:

- A client would have to ask many nodes to find where a file's blocks are, which is slow.
- Keeping copies of the namespace consistent across nodes would need constant coordination.
- Losing a DataNode could lose part of the file system map, not just data.
- Data placement and re-replication decisions would have no single coordinator.

Q2. Describe the default block replication placement policy and why it uses multiple racks.

With replication factor 3:

1. First replica on the writer's node (or a random node in the client's rack).
2. Second replica on a node in a different rack.
3. Third replica on another node in the same rack as the second.

Reasons:

- Durability: losing a whole rack (switch or power failure) cannot destroy every copy.
- Performance: two replicas share a rack, so write traffic crosses racks only once, and reads can often stay in-rack.

The trade-off is extra cross-rack bandwidth on writes in exchange for surviving rack failure.

Q3. How does the YARN ResourceManager decide which application gets resources? Discuss FIFO, Capacity and Fair schedulers.

The ResourceManager uses a pluggable scheduler. Applications ask for containers (memory, vCores, locality preference); the scheduler decides who gets what based on its policy.

Scheduler	How it works	Good for	Weakness
FIFO	First submitted, first served	Single-user or test clusters	A big job blocks all later jobs
Capacity	Queues get guaranteed shares (e.g. Finance 40%, ML 60%); idle capacity can be borrowed	Multi-team clusters with guarantees	Needs queue planning
Fair	Resources are shared out so active apps get roughly equal shares over time	Mixed short and long jobs	Long jobs may run slower

Most production clusters use Capacity or Fair, sometimes with preemption to reclaim borrowed resources.

Q4. Distinguish the roles of the ResourceManager, NodeManager and ApplicationMaster.

Component	Count	Role
ResourceManager	One per cluster (plus standby for HA)	Global scheduling and container allocation; tracks cluster capacity; starts each app's first container
NodeManager	One per worker node	Launches and monitors containers on its node, reports health and usage by heartbeat, aggregates logs
ApplicationMaster	One per application	Negotiates containers from the RM, places tasks (with data locality), monitors and retries failed tasks

Short version: the RM decides how much, the AM decides what runs where for its own job, and the NM actually runs it.

Q5. Explain the shuffle-and-sort phase. Why is it usually the most expensive part of a MapReduce job?

After the map phase, all values for the same key must reach the same reducer.

1. Each mapper partitions its output by key (default `hash(key) mod R`), sorts it, and spills it to local disk.
2. Each reducer pulls its partition from every mapper over the network.
3. The reducer merges and sorts the pieces by key.
4. Values for each key are grouped and passed to the reduce function.

It is expensive because it combines disk writes, network transfer from every mapper to every reducer, sorting, and a barrier (reducers cannot finish until all maps finish). Combiners, compression of map output and good partitioning reduce the cost.

Q6. Compare map tasks and reduce tasks across input, output, data movement and fault behaviour.

Aspect	Map task	Reduce task
Input	One input split (usually one HDFS block)	Its partition of intermediate data from all mappers
Output	Intermediate key-value pairs on local disk	Final output written to HDFS
Data movement	Mostly local read (data locality)	Network-heavy shuffle
Number	One per split, decided by input size	Set by the user
Fault behaviour	Rerun on another node; its input is still in HDFS	Rerun and refetch map outputs; may force maps to rerun if their node died

Q7. What distinguishes a combiner from a reducer? Give an example where a combiner would be unsafe.

A combiner is an optional mini-reduce that runs on each mapper's output before the shuffle to cut network traffic. A reducer runs after the shuffle over all values for a key from all mappers.

- The framework may run a combiner zero, one or many times, so the result must not depend on it.
- It is only safe when the operation is commutative and associative (sum, count, max, min).

Unsafe example: average. If mapper A has values 1, 2, 3 (avg 2) and mapper B has 10 (avg 10), averaging the averages gives 6, but the true average is 4. Fix: emit (sum, count) pairs and divide only in the reducer.

Q8. How does HDFS fault tolerance differ from YARN's? Which failures does each handle?

Layer	Protects	Failures handled	Mechanism
HDFS	Data	DataNode crash, disk failure, corrupted block, NameNode failure (with HA)	Block replication, checksums, heartbeats, re-replication, standby NameNode with JournalNodes
YARN	Running work	NodeManager crash, container failure, ApplicationMaster failure, RM failure (with HA)	Heartbeats, container reallocation, AM restart, standby ResourceManager

MapReduce adds a third layer on top: task retry and speculative execution. HDFS keeps the data safe; YARN keeps compute running.

Q9. Contrast reduce-side join and map-side join. When would you choose each?

	Reduce-side join	Map-side (replicated) join
How	Mappers tag each record with its source and emit the join key; matching records meet at a reducer	Small table is loaded into memory on every mapper; large table is joined as it streams through
Shuffle	Full shuffle of both datasets	None for the large table
Works when	Both datasets are large	One dataset fits in memory
Cost	High network and disk	Low, but memory per mapper

Choose map-side when one side is small and stable (e.g. a product lookup table). Choose reduce-side when both sides are large (e.g. orders joined with clickstream).

Q10. Why is YARN's separation of resource management from processing logic important for multi-tenant clusters?

Before YARN (Hadoop 1), the MapReduce JobTracker did both scheduling and resource management, so only MapReduce could run well on the cluster.
With YARN:

- MapReduce, Spark, Tez and others share one cluster and one copy of the data.
- Each application brings its own ApplicationMaster with its own logic.
- Queues and schedulers give teams guaranteed or fair shares.
- Containers isolate memory and CPU so one tenant cannot starve another.
- Utilisation goes up because idle capacity is shared.

Q11. Word Count on 1 TB: one reducer is far slower than the others. Diagnose and give two mitigations.

Cause: data skew. Word frequencies follow a Zipf-like distribution, so a few words ("the", "a", "of") produce a huge share of the pairs. With hash partitioning, every pair for "the" goes to the same reducer, which becomes a straggler.
Mitigations (any two):

- Use a combiner so each mapper sends one (word, partial_count) per word instead of millions of (word, 1).
- Salt hot keys: emit ("the#3", 1) with a random suffix, aggregate per salted key, then sum the partial results in a second step.
- Use a custom partitioner that spreads known hot keys over several reducers.
- Filter stop words if they are not needed in the result.

Speculative execution does not fix skew, because the duplicate task has the same oversized input.

Q12. A job processes many 10 KB files and the NameNode runs into memory problems. Why, and how do you fix it?

Every file, directory and block is an object in NameNode memory (roughly 150 bytes each as a rule of thumb). A 10 KB file still costs at least one file object and one block object, so millions of tiny files consume NameNode heap while storing very little data. It also creates one map task per small file, so task start-up overhead dominates.

Fixes:

- Merge small files into large ones before or after loading (e.g. a compaction job).
- Store them in container formats: SequenceFiles, HAR (Hadoop Archives), or columnar files like Parquet/ORC.
- Use CombineFileInputFormat so one map task reads many small files.
- Fix the producer to write larger batches.

Q13. MapReduce batch jobs and Spark analytics share a cluster. How does YARN keep both fair, and what configuration matters?

Both run as YARN applications with their own ApplicationMasters and request containers from the same ResourceManager. Fairness comes from the scheduler.

Configuration that matters:

- Scheduler type (Capacity or Fair) and queue definitions, e.g. a `batch` queue and an `analytics` queue with guaranteed shares.
- Maximum capacity per queue so one queue cannot take everything.
- User limits within a queue.
- Preemption to reclaim borrowed resources when the owning queue needs them.
- Container sizes (min/max memory and vCores) and NodeManager capacity so containers fit the hardware.
- For Spark, dynamic allocation so idle executors are released back to YARN.

Q14. A map task fails repeatedly on the same node. What does the framework do, and what should an administrator do?

Framework:

- The ApplicationMaster retries the task, normally on a different node (default up to 4 attempts per task).
- If a node causes repeated failures, the AM blacklists it for that job.
- If all attempts fail, the job fails.
- The NodeManager health checker can mark the node unhealthy so the RM stops scheduling there.

Administrator:

- Read container logs (log aggregation, `yarn logs –applicationId ...`).
- Check the node: disk full or failing, bad memory, wrong JDK or library version, misconfiguration.
- Decommission or repair the node, then recommission it.
- Confirm HDFS has re-replicated any blocks from failed disks.

Q15. A data scientist wants interactive queries on the data your MapReduce cluster processes. Is HDFS + MapReduce suitable? What would you recommend?

HDFS is fine as storage, but MapReduce is not suitable for interactive work. Each query becomes a job with container start-up, disk writes between stages and a full shuffle, so latency is minutes, not seconds.

Recommended:

- Spark SQL (in-memory, Catalyst optimizer) or Trino/Presto or Impala for SQL queries in seconds.

- Hive on Tez with LLAP if the team already uses Hive.
- Store curated data as partitioned Parquet or ORC so queries read only needed partitions and columns.
- Pre-aggregate common results into small tables for dashboards.

Keep MapReduce (or Spark batch) for heavy scheduled ETL.

Worked example: block count and raw storage

File size S = 1 GB = 1024 MB, block size B = 128 MB, replication R = 3.

- Blocks: ceil(1024 / 128) = 8 blocks.
- Block replicas stored: 8 x 3 = 24.
- Raw storage: 1024 MB x 3 = 3072 MB = 3 GB.
- NameNode tracks 1 file object + 8 block objects.

Same question for a 300 MB file: ceil(300 / 128) = 3 blocks (128 + 128 + 44 MB). Raw storage is 300 x 3 = 900 MB, not 3 x 128 x 3, because the last block only uses the space it needs.

Worked example: Word Count step by step

Input, two documents (each becomes one split and one map task):

- doc1: `the cat sat on the mat`
- doc2: `the dog sat on the log`

1. Map (doc1): (the,1) (cat,1) (sat,1) (on,1) (the,1) (mat,1)
2. Map (doc2): (the,1) (dog,1) (sat,1) (on,1) (the,1) (log,1)
3. Combine (optional, per mapper):
    - doc1: (the,2) (cat,1) (sat,1) (on,1) (mat,1)
    - doc2: (the,2) (dog,1) (sat,1) (on,1) (log,1)
4. Partition: each key goes to reducer `hash(key) mod R`. With R = 2, say "the", "sat", "mat", "log" go to reducer 0 and the rest to reducer 1.
5. Shuffle and sort: reducer 0 receives, sorted by key: log:[1], mat:[1], sat:[1,1], the:[2,2].
6. Reduce: sum the list for each key.
7. Output (two part files in HDFS):

```
part-r-00000: log 1, mat 1, sat 2, the 4
part-r-00001: cat 1, dog 1, on 2
```

Note the combiner cut the pairs sent over the network from 12 to 10. On real data the saving is much larger.

Quick recall

- NameNode = metadata in memory; DataNodes = blocks on disk.
- Default block size 128 MB, default replication 3.
- Placement: local node, then a remote rack, then another node on that same remote rack.
- Data never flows through the NameNode.
- YARN: RM schedules cluster, NM runs containers, AM runs one application.
- MapReduce flow: split, map, (combine), partition, shuffle and sort, reduce, output.
- Default partitioner: hash(key) mod number of reducers.
- Map output goes to local disk; final output goes to HDFS.
- Combiners only for commutative and associative operations; never average directly.
- Map-side join needs one small table; reduce-side join handles two large tables.

- Small files hurt the NameNode; merge them or use container formats.
- Skew is fixed with combiners, salting or custom partitioners, not speculative execution.

Day 3: Apache Hive and Apache Pig

Answers to the 15 Day 3 Knowledge Review questions, with short HiveQL and Pig Latin examples.

Q1. Explain schema-on-read. How does it differ from schema-on-write, and what trade-offs does it bring?

With **schema-on-read**, files are stored as they arrive and the table schema is applied only when a query reads them. Hive keeps the schema in its metastore and uses a SerDe to interpret the bytes. There is still a schema; it is just checked late.

	Schema-on-write (RDBMS)	Schema-on-read (Hive)
When schema is checked	At insert	At query time
Load speed	Slower (validate, convert)	Fast (copy files as-is)
Query speed	Faster (data already clean)	Slower (parse every read)
Bad data	Rejected at load	Shows up as NULLs or wrong values at query time
Schema change	Migration needed	New table definition over the same files

Trade-off: flexible and fast ingestion of messy, changing data, paid for with read-time parsing cost and later discovery of data quality problems.

Q2. Describe the role of the Hive metastore. What happens if it is unavailable?

The metastore is Hive's catalog. It stores databases, tables, columns and types, partitions and their locations, the storage format and SerDe, and statistics. It is usually backed by a relational database (MySQL or PostgreSQL in production, Derby only for testing). It does not store the data files.

During semantic analysis the compiler asks the metastore whether tables and columns exist and which partitions to read.

If it is down:

- New queries fail at compile time, because Hive cannot resolve table names or file locations.
- DDL (CREATE, ALTER, ADD PARTITION) fails.
- Other tools sharing it (Spark SQL, Trino) also lose their table definitions.
- The data in HDFS is safe; only access through table names is lost.

Production setups run several metastore services against a replicated database to avoid this single point of failure.

Q3. Explain the difference between managed and external tables. Why does it matter for the data lifecycle?

	Managed (internal)	External
Who owns files	Hive	Another system or team
Location	Hive warehouse directory	Given with `LOCATION`
`DROP TABLE`	Deletes metadata and data files	Deletes metadata only; files stay
Use when	Hive creates and fully controls the data	Data is shared, produced by other jobs, or must survive a drop

```
CREATE EXTERNAL TABLE raw_sales (
  sale_id BIGINT, product_id BIGINT, amount DOUBLE
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
LOCATION '/data/landing/sales';
```

It matters because a mistaken `DROP TABLE` on a managed table destroys the data. External tables protect shared data from that.

Q4. Explain partitioning and bucketing. Give a query that benefits from each.

Partitioning stores each value of a partition column in its own directory (`dt=2024-03-15/region=North/`). A filter on that column lets Hive skip every other directory (partition pruning).

Bucketing hashes a column into a fixed number of files inside each partition. Rows with the same key always land in the same bucket file, which helps joins and sampling.

```
CREATE TABLE sales (
  sale_id BIGINT, product_id BIGINT, store_id BIGINT, amount DOUBLE
)
PARTITIONED BY (dt STRING, region STRING)
CLUSTERED BY (product_id) INTO 32 BUCKETS
STORED AS ORC;
```

- Benefits from partitioning: `SELECT SUM(amount) FROM sales WHERE dt = '2024-03-15';` reads one day, not the whole year.
- Benefits from bucketing: a join of `sales` with `products` on `product_id`, when both are bucketed on `product_id` with compatible bucket counts, can run as a bucket map join without a full shuffle. Also `TABLESAMPLE(BUCKET 1 OUT OF 32 ON product_id)` reads about 1/32 of the data.

Keep partition columns low to moderate in cardinality; too many partitions overload the metastore.

Q5. Describe row-oriented vs column-oriented storage. Give a workload where each fits.

	Row-oriented (Text, CSV, Avro)	Column-oriented (Parquet, ORC)
Layout	All fields of a row together	All values of a column together
Reads only needed columns	No	Yes (projection)
Compression	Moderate	High (similar values sit together)
Skip data with min/max stats	No	Yes (predicate pushdown)
Good for	Writing or reading whole records	Scans, filters and aggregations on a few columns

- Row-oriented fit: an ETL step or event stream (Kafka to Avro) that processes every field of each record.
- Column-oriented fit: `SELECT region, SUM(amount) FROM sales GROUP BY region` on a 200-column table; only two columns are read.

Q6. Compare Hive and Pig across abstraction, schema handling and typical user.

	Hive	Pig
Language	HiveQL, declarative and SQL-like	Pig Latin, procedural dataflow
You describe	The result you want	The steps to get it
Schema	Declared once in the metastore, shared across tools	Declared in each script with `LOAD ... AS (...)`, optional
Data model	Tables, partitions, buckets	Relations of tuples, bags, maps
Typical user	Analysts, BI users who know SQL	Data engineers writing ETL pipelines
Best for	Reporting and ad hoc analysis	Multi-step transformations, nested data

Both compile to jobs on the same cluster engine (MapReduce, Tez, or Spark depending on version and config), so the choice is mostly about authoring style and team skills.

Q7. What are the main categories of UDFs in Hive, and what does each do?

Type	Input to output	Example
UDF (scalar)	One row, one value	`mask_phone(phone)`
UDAF (aggregate)	Many rows, one value per group	A custom weighted median
UDTF (table-generating)	One row, many rows	`explode(tags)` turns an array into rows

Use built-in functions first; custom UDFs must be packaged as a jar, registered, deployed to every node and maintained, and the optimizer treats them as black boxes. Keep them deterministic.

```
ADD JAR /libs/my-udfs.jar;
CREATE TEMPORARY FUNCTION mask_phone AS 'com.example.MaskPhone';
SELECT mask_phone(phone) FROM customers;
```

Q8. Explain the role of SerDes. How does a SerDe interact with the metastore schema?

A SerDe (serializer/deserializer) converts between bytes in a file and the row objects Hive works with. The deserializer is used on read, the serializer on write.

The metastore holds two things per table: the schema (column names and types) and which SerDe to use. At read time the SerDe parses each record and maps its fields onto the declared columns. The same bytes can mean different things under a different SerDe or schema.

Example: the line `2024-03-15|North|450.00` reads as three columns with a delimited text SerDe using `|`, but fails or returns NULLs with a JSON SerDe. Common SerDes: LazySimpleSerDe (delimited text), OpenCSVSerde, JsonSerDe, RegexSerDe, plus Avro, Parquet and ORC SerDes.

Q9. Compare HiveQL and Pig Latin for a multi-stage ETL pipeline. Which is more natural, and why?

Pig Latin is usually more natural for multi-stage ETL. Each step is a named relation, so cleaning, branching (`SPLIT`), joining and reshaping read top to bottom like a pipeline, and nested types (bags, tuples, maps) need no flattening first. In HiveQL the same pipeline becomes nested subqueries or several intermediate tables.

Same aggregation, revenue per category for one day:

```
-- HiveQL
SELECT p.category, SUM(s.amount) AS revenue
FROM sales s JOIN products p ON s.product_id = p.product_id
WHERE s.dt = '2024-03-15'
GROUP BY p.category;
```

```
-- Pig Latin
sales    = LOAD '/data/sales' USING PigStorage(',')
           AS (sale_id:long, product_id:long, amount:double, dt:chararray);
products = LOAD '/data/products' USING PigStorage(',')
           AS (product_id:long, name:chararray, category:chararray);
day      = FILTER sales BY dt == '2024-03-15';
joined   = JOIN day BY product_id, products BY product_id;
by_cat   = GROUP joined BY products::category;
revenue  = FOREACH by_cat GENERATE group AS category, SUM(joined.day::amount) AS revenue;
STORE revenue INTO '/out/revenue_by_category' USING PigStorage(',');
```

For a single report the HiveQL is shorter and readable by any SQL user; for a long chain of transformations the Pig script is easier to follow and debug step by step.

Q10. Why is Hive not suited to interactive, low-latency workloads? Which design decisions cause this?

- Every query is compiled into distributed jobs; start-up and scheduling alone can take seconds.
- It was built for full scans of large data, optimising throughput, not response time.

- Schema-on-read means parsing and deserialising data at query time.
- There are no row-level indexes for point lookups like an OLTP database.
- Joins and aggregations shuffle data across the network, with intermediate results written to disk (especially on MapReduce).
- Data lives in HDFS files, not in a memory-resident store.

Hive on Tez with LLAP caching and columnar formats can bring some queries down to seconds, but sub-second, high-concurrency workloads belong on tools such as Trino/Presto, Impala, Druid or a relational database.

Q11. A Hive table has 500 partitions, and a query filters on a partition column whose values are not in the file paths. Diagnose and fix.

Likely problem: the partition metadata and the directory layout do not match. Partition columns are not stored inside the data files; Hive gets their values from the metastore and the `col=value` directory names. If the files were written straight into the table folder (not into `dt=.../` subfolders), or partitions were never registered, Hive either finds nothing or scans everything, and pruning does not work.
Fix:

- Lay the files out as `/table/dt=2024-03-15/...` so the directory name carries the value.
- Register partitions: `MSCK REPAIR TABLE sales;` or `ALTER TABLE sales ADD PARTITION (dt='2024-03-15') LOCATION '...';`
- Or reload with a dynamic partition insert so Hive creates the layout:

```
SET hive.exec.dynamic.partition.mode=nonstrict;
INSERT OVERWRITE TABLE sales PARTITION (dt)
SELECT sale_id, product_id, store_id, amount, dt FROM raw_sales;
```

- Write the filter directly on the column (`WHERE dt = '2024-03-15'`), not wrapped in a function, so pruning can apply. Check with `EXPLAIN` and `SHOW PARTITIONS sales`.

Q12. A Pig script produces a very large intermediate relation between two joins. How do you tell whether shuffle volume or skew is the cause, and what do you try first?

Diagnose:

- Run `EXPLAIN` to see the physical plan and how many jobs and shuffles there are.
- Use `DESCRIBE` and `ILLUSTRATE` on a sample to see the shape and size of each relation.
- Check job counters: map output records and bytes show shuffle volume.
- Compare reduce task durations. If one or two reducers run far longer than the rest, it is skew. If all are slow and evenly loaded, it is volume.

First fixes:

- Reduce volume early: `FILTER` before the joins, and `FOREACH ... GENERATE` only the needed fields.
- If one side is small, use `JOIN ... USING 'replicated'` to avoid the shuffle.
- If it is skew on a hot key, use `JOIN ... USING 'skewed'`.
- Check for a key that explodes the join (many-to-many or NULL-heavy key) and fix the join key.

Q13. Queries on an inherited Hive table return strange NULLs in some columns. Where do you look first, and what does it suggest?

Look at the table definition first: `SHOW CREATE TABLE t;` or `DESCRIBE FORMATTED t;`, then compare it with a few raw lines from the files (`hdfs dfs -cat /path/part-00000 | head`).

It suggests a schema or SerDe mismatch, because under schema-on-read Hive returns NULL when a field cannot be parsed into the declared type. Typical causes:

- Wrong field delimiter, so fields shift or merge.
- Column order in the DDL differs from the file.
- Type mismatch (text like `"1,200"` declared as INT, odd timestamp formats).
- Schema evolved: older files lack new columns.
- Wrong SerDe (CSV read with LazySimpleSerDe, quoted fields not handled).
- Header row not skipped (`skip.header.line.count`).

Q14. Hive or Pig for a pipeline that ingests heterogeneous JSON, flattens nested fields and writes Parquet? Why?

Recommend Pig for the transformation itself. JSON with nested and varying structure maps naturally to Pig's tuples, bags and maps, and a step-by-step script (load, clean, `FLATTEN`, split good and bad records, store) is easier to write and debug than nested HiveQL with `LATERAL VIEW explode`.

Hive is still a fair choice if the team only knows SQL: a JSON SerDe plus `LATERAL VIEW explode(...)` can flatten the data, and `INSERT ... STORED AS PARQUET` writes the output. Either way, register the final Parquet as an external Hive table so analysts can query it. In many teams today the same job would be done in Spark.

Q15. An analyst asks why Hive queries take minutes, not seconds. Explain, and say what can and cannot be done.

Why: each query compiles into distributed jobs with start-up cost, scans large files from HDFS, parses data at read time, and shuffles data between stages. Hive was designed for throughput on large batch scans, not for fast single answers.

What can be done:

- Use Tez (or Spark) instead of MapReduce, and LLAP caching where available.
- Store data as ORC or Parquet with compression.
- Partition by common filters (date) and filter on partition columns.
- Select only needed columns; avoid `SELECT *`.
- Keep table statistics up to date (`ANALYZE TABLE ... COMPUTE STATISTICS`) and enable cost-based optimisation.
- Use map joins for small tables, bucketing for large joins.
- Pre-aggregate into small summary tables for dashboards.

What cannot reasonably be done: make Hive an OLTP or sub-second, high-concurrency engine. For that, serve results from Trino/Presto, Impala, Druid, or a relational database fed by Hive.

Quick recall

- Hive is a SQL query layer over files; the engine underneath is MapReduce, Tez or Spark.
- Schema-on-read: schema exists, checked when reading, not loading.
- Metastore holds metadata only; backed by MySQL or PostgreSQL in production.
- Dropping a managed table deletes data; dropping an external table keeps it.
- Partition = directory per value, enables pruning. Bucket = hash into fixed files, helps joins and sampling.
- Partition columns are not stored inside the data files.
- Parquet and ORC are columnar: read fewer columns, compress better, skip blocks.
- SerDe converts bytes to rows and back; wrong SerDe or delimiter gives NULLs.
- UDF one-to-one, UDAF many-to-one, UDTF one-to-many (e.g. `explode`).
- Pig Latin is procedural; a bag is an unordered collection of tuples.
- Pig join options: `replicated` for small tables, `skewed` for hot keys.

- Hive suits batch analytics; it is not built for sub-second queries.

Day 3 Assignment: Answers

Correct answers to the 15 Day Three assignment questions on Hive and Pig, each with a one-line reason.

Q1. Schema-on-read means the data has no schema.

False. The schema exists in the metastore; Hive applies it when the data is read, not when it is loaded.

Q2. Why are columnar formats like Parquet/ORC preferred for analytical queries?

b. They enable column projection and better compression. A query reads only the columns it needs, and values from one column compress well together.

Q3. Partitioning and bucketing are the same thing.

False. Partitioning splits data into directories by value; bucketing hashes rows into a fixed number of files.

Q4. Bucketing in Hive writes rows into a fixed number of files based on:

c. Hash of a column. The same value always lands in the same bucket file.

Q5. Which UDF type produces many output rows per input row (e.g. splitting a string)?

d. UDTF. One row in, many rows out, as with `explode()`.

Q6. Match each concept to its description.

Concept	Description
Pig Latin	Dataflow scripting language
UDTF	Produces many output rows per input row
Metastore	Central catalog of table and partition definitions
Managed table	Hive owns and drops the underlying data files

Managed versus external comes down to who owns the files.

Q7. What is the role of a SerDe in Hive?

d. Converts between byte streams and rows. SerDe is short for serializer and deserializer.

Q8. What does schema-on-read mean in Hive?

c. Schema is applied when data is read. Traditional databases check the schema on write instead.

Q9. Which type of database is most commonly used as the backend for the Hive metastore?

A relational database (for example MySQL or PostgreSQL). The built-in Derby supports only one user at a time, so it is used for testing.

Q10. Hive partitioning primarily enables:

b. Partition pruning (skip unrelated directories). It only helps when the query filters on the partition column.

Q11. Which statement about Hive's execution engine is correct?

a. Hive is a query layer that can use MapReduce, Tez, or Spark. The engine is set with `hive.execution.engine` (Spark support was removed in Hive 4, so check your version).

Q12. Which Pig complex type is an unordered collection of tuples?

a. Bag. A tuple is an ordered set of fields; a bag is an unordered group of tuples.

Q13. Which statement best describes Pig Latin's programming style?

d. Procedural, dataflow-oriented. A Pig script lists the steps; SQL only states the result you want.

Q14. What does the Hive metastore store?

b. Table, column and partition metadata. The data files themselves stay in HDFS.

Q15. What happens when you DROP an external Hive table?

c. Only metadata is removed; files stay. Dropping a managed table would delete the files too.

Quick recall

- Schema-on-read still has a schema; it is applied at query time.
- Columnar formats: read fewer columns, better compression.
- Partitioning uses directories by value; bucketing uses hash into a fixed number of files.
- UDTF: one input row, many output rows.
- SerDe: bytes to rows and rows to bytes.
- Metastore stores metadata in a relational database; Derby is for testing.
- Partition pruning only works when you filter on the partition column.
- Hive is a query layer on MapReduce, Tez or Spark.
- Pig bag: unordered collection of tuples.
- Pig Latin is procedural dataflow.
- DROP external table: metadata gone, files kept.

Day 4: Apache Spark, Knowledge Review Answers

Answers to the 15 review questions at the end of the Day 4 slides (Spark architecture, RDDs, DataFrames, Spark SQL, optimization, MLlib and GraphX).

Conceptual questions

Q1. Explain why Apache Spark was developed. What specific limitations of disk-oriented batch processing did it address?

Spark was built to make **iterative** and multi-step jobs fast. MapReduce writes every intermediate result to disk and starts a new job for each step, so a machine learning algorithm that loops 50 times reads and writes the same data 50 times.

Spark addressed three limits:

- **Repeated disk I/O.** Spark can keep a dataset in memory across steps (`cache()`), so each loop reads RAM, not disk.
- **Job overhead and chaining.** In MapReduce the programmer chains many jobs by hand. In Spark one program describes the whole pipeline and the engine splits it into stages.
- **Limited programming model.** MapReduce only offers map and reduce. Spark adds joins, filters, SQL, machine learning and graph processing in one engine.

Fault tolerance comes from lineage (recompute lost parts) instead of writing every step to replicated storage.

Q2. Distinguish the driver, executors, and the cluster manager. What happens if the driver dies? What happens if an executor dies?

Part	Role
Driver	Runs your `main` program, builds the DAG, splits it into stages and tasks, schedules tasks, collects results
Executors	Processes on worker nodes that run tasks and hold cached data and shuffle files
Cluster manager	Gives out CPU and memory (YARN, Kubernetes, Standalone, or local mode)

- **Driver dies:** the whole application fails. All its state (the DAG, task tracking, cached data references) is lost. You must restart the job. (In YARN cluster mode the ApplicationMaster can retry the driver, but it starts from the beginning.)
- **Executor dies:** the application survives. The driver reruns the lost tasks on other executors, and recomputes any lost cached or shuffle data from lineage. The job is slower but finishes.

Q3. Explain the difference between transformations and actions. Why does Spark defer execution until an action is called?

A **transformation** (`map`, `filter`, `join`, `groupBy`) returns a new RDD or DataFrame and only records the step. An action (`count`, `collect`, `show`, `write`) returns a result or writes data, and it is the only thing that starts a job. Spark waits (lazy evaluation) because seeing the whole chain first lets it:

- fuse many narrow steps into one stage, so data is read once;
- push filters and column selection down to the data source;
- skip work whose result is never used.

```
df = spark.read.parquet("sales")          # nothing runs
big = df.filter(df.amount > 500)          # nothing runs
big.count()                               # job starts here
```

Q4. Distinguish narrow from wide transformations. Give an example of each and explain why the difference matters.

- **Narrow:** each output partition needs data from only one input partition. Examples: `map`, `filter`, `flatMap`, `withColumn`. No data moves between machines.
- **Wide:** an output partition needs data from many input partitions. Examples: `groupByKey`, `reduceByKey`, `join`, `distinct`, `repartition`. Data must be **shuffled** across the network.

Why it matters: every wide transformation ends a stage. A shuffle writes data to disk, sends it over the network and waits for all upstream tasks, so it is usually the most expensive part of a job. Fewer and smaller shuffles mean faster jobs. In `explain()` output a shuffle shows up as `Exchange`.

Q5. Explain how Spark recovers from a lost partition without replicating intermediate data.

Through **lineage**. Every RDD remembers its parent and the function that built it. If a partition is lost (executor crash), Spark walks back through the lineage to the nearest data that still exists (the source file in HDFS, a cached partition, or surviving shuffle files) and reruns only the steps needed for that one partition.

- Narrow dependencies: recompute just the matching parent partition. Cheap.
- Wide dependencies: may need the shuffle output of many parent partitions; if those shuffle files are also lost, the earlier stage reruns.

When lineage gets very long (iterative algorithms), use `checkpoint()` to save the data to reliable storage and cut the lineage.

Comparative questions

Q6. Compare RDDs, DataFrames, and Datasets across schema, type safety, optimization, and language support.

	RDD	DataFrame	Dataset
Schema	None (opaque objects)	Yes (named, typed columns)	Yes
Type safety	Compile-time for the objects (Scala/Java)	Checked at runtime (column names are strings)	Compile-time (Scala/Java)
Optimization	None by Catalyst	**Catalyst** + Tungsten	Catalyst + Tungsten
Languages	Scala, Java, Python	Scala, Java, Python, R, SQL	Scala, Java only
Use it for	Custom low-level logic, unstructured data	Most analytics and ETL	Typed Scala/Java code

A DataFrame is a `Dataset[Row]` in Scala. In Python you only have RDDs and DataFrames.

Q7. Explain the role of the Catalyst optimizer. What rewrites does it apply, and what does it need in order to apply them?

Catalyst turns a DataFrame or SQL query into an efficient physical plan. It goes through four phases: analysis (resolve names and types against the schema or catalog), logical optimization (rule-based rewrites), physical planning (pick join and aggregation strategies, using cost estimates), and code generation (Tungsten whole-stage codegen).
Common rewrites:

- predicate pushdown (filter before join, or inside the Parquet reader);
- column pruning (read only needed columns);
- constant folding (`1 + 2` becomes `3`);
- choosing broadcast join instead of sort-merge join when one side is small;
- combining adjacent filters and projections.

What it needs: a **schema** and declarative operations it can see inside. It cannot optimize what it cannot read, so an RDD lambda or a Python UDF is a black box to it.

Q8. Compare broadcast joins and shuffle joins. When is each appropriate?

	Broadcast hash join	Shuffle join (sort-merge or shuffle hash)
How	Small table is copied to every executor; big table stays where it is	Both tables are shuffled so equal keys meet in the same partition
Shuffle of the big table	None	Yes
Memory	Small side must fit in each executor	Works for any size, spills to disk
Use when	One side is small (lookup or dimension table)	Both sides are large

Spark broadcasts automatically when a side is below `spark.sql.autoBroadcastJoinThreshold` (10 MB by default). You can force it with a hint:

```
from pyspark.sql.functions import broadcast
sales.join(broadcast(products), "product_id")
```

Broadcasting a table that is not really small can cause out-of-memory errors on the driver or executors.

Q9. Explain the difference between groupByKey and reduceByKey. Which is preferred, and why?

`groupByKey` shuffles **every value** across the network and then builds a list per key. `reduceByKey` first combines values inside each partition (a map-side combine, like a MapReduce combiner) and only shuffles one partial result per key per partition.

`reduceByKey` is preferred for aggregations: much less shuffle data, less memory, and no risk of one key's huge list not fitting in memory. Use `groupByKey` only when you really need all the values (for example, to sort them).

```
pairs = words.map(lambda w: (w, 1))
counts = pairs.reduceByKey(lambda a, b: a + b)          # preferred
counts2 = pairs.groupByKey().mapValues(len)             # same result, more shuffle
```

The function given to `reduceByKey` must be associative and commutative (sum, max, min), because Spark applies it in any order.

Q10. Distinguish the DataFrame-based spark.ml API from the older RDD-based spark.mllib API. Which is the primary API for new projects?

- `spark.ml`: works on DataFrames, uses Transformers, Estimators and Pipelines, benefits from Catalyst, gets all new features. This is the **primary API**.
- `spark.mllib`: works on RDDs of vectors and `LabeledPoint`. It has been in maintenance mode since Spark 2.0 (bug fixes only).

New projects should use `spark.ml`. "MLlib" is still the name of the library as a whole.

Scenario-based questions

Q11. A Spark job takes two hours, and one task in a shuffle stage runs ten times longer than the others. Diagnose the problem and propose two mitigations.

Diagnosis: **data skew**. One key (often a very common value, or `NULL`) holds far more rows than the others, so the partition that receives it does most of the work while the other tasks finish and wait. Confirm in the Spark UI: open the stage, look at the task summary; max duration and shuffle read size are far above the median.

Mitigations:

1. **Salting:** add a random number to the hot key so its rows spread over many partitions, aggregate, then aggregate again without the salt.

```
from pyspark.sql import functions as F
N = 8
salted = df.withColumn("salt", (F.rand() * N).cast("int"))
partial = salted.groupBy("key", "salt").agg(F.sum("amount").alias("s"))
result = partial.groupBy("key").agg(F.sum("s").alias("total"))
```

2. **Adaptive Query Execution skew join:** with `spark.sql.adaptive.enabled` and `spark.sql.adaptive.skewJoin.enabled` (both on by default in Spark 3.2+), Spark splits oversized join partitions at runtime.
3. Other options: broadcast the smaller side if possible, filter out or handle `NULL` keys separately, or process the few hot keys in a separate job and `union` the results.

Q12. You join two very large DataFrames and observe a heavy shuffle. What do you check first, and what changes might reduce the shuffle cost?

Check first:

- Is one side really large, or could it be broadcast after filtering?
- Are filters and column selection applied before the join? (Look at `explain()`: are the filters below the `Exchange`?)
- Is the shuffle balanced, or is one key skewed (Spark UI task sizes)?
- Do the join key types match on both sides (a string key joined to an int adds casts and can block optimization)?
- Is `spark.sql.shuffle.partitions` (default 200) sensible for the data size?

Changes that reduce shuffle cost:

- filter early and `select` only the needed columns;
- pre-aggregate one side before joining if the result only needs totals;
- broadcast a side that becomes small after filtering;
- if the same large tables are joined again and again, write them **bucketed** on the join key (`bucketBy`) so later joins skip the shuffle;
- let AQE coalesce small partitions and handle skew.

Q13. A team trains a model on data that fits comfortably on one machine. Would Spark be the right choice? Explain.

Usually no. On one machine, tools like pandas, scikit-learn, XGBoost or PyTorch are faster to run, faster to iterate with, and have more algorithms. Spark adds overhead: job scheduling, serialization between JVM and Python, and network coordination, with no benefit when the data already fits in RAM.

Spark becomes the right choice when:

- the data will grow beyond one machine soon;
- the features are already produced by a Spark pipeline and moving data out is costly;
- you need to run many trainings in parallel (for example, a large hyperparameter search).

Q14. An analyst wants sub-second response time on a 10 GB dataset. Is Spark appropriate? What alternatives would you recommend?

Not a good fit. Spark is built for throughput, not latency. Each query pays for planning, job scheduling and task launch, which is often hundreds of milliseconds to several seconds even on cached data.

Better options:

- **DuckDB** or a single-node columnar engine: 10 GB fits on one good machine and queries return in milliseconds.
- An OLAP store built for interactive queries: ClickHouse, Apache Druid, Apache Pinot.
- A distributed SQL engine for ad hoc queries over a data lake: Trino or Presto.
- Use Spark to precompute aggregates on a schedule and serve the small results from a database or cache.

Q15. You must analyze a social graph with 500 million users and 5 billion edges. Which Spark library would you use, and what design considerations would guide the implementation?

Use **GraphX** (Scala or Java), or GraphFrames for a DataFrame and Python API. Both offer PageRank, connected components and triangle counting. GraphX itself gets few new features now, so check the Spark version you run. Design considerations:

- **Partitioning:** choose a good edge partition strategy (for example `EdgePartition2D`) to cut communication.
- **High-degree vertices:** celebrity accounts with millions of followers cause skew; consider special handling or sampling.
- **Memory:** 5 billion edges need a large cluster; store graph data as Parquet and size executors for it.
- **Iterations:** each Pregel superstep is a full distributed round; set a max number of iterations or a convergence tolerance.
- **Checkpointing:** checkpoint every few iterations to keep lineage short.
- **Join back:** results (rank, component id) are joined back to user attributes with Spark SQL for reporting.

How to read `explain()` output

Run `df.explain()` (or `df.explain(True)` for all plan levels) and read the physical plan from the bottom up.

You see	It means
`FileScan parquet`	Reading the source files
`ReadSchema: struct<a,b>`	Only these columns are read (column pruning worked)
`PushedFilters: [GreaterThan(amount,500)]`	Filter sent into the file reader; Parquet/ORC can skip row groups
`PartitionFilters: [dt = 2024-03-15]`	Whole partition folders skipped (partition pruning)
`Exchange hashpartitioning(key, 200)`	A **shuffle**: a stage boundary
`BroadcastExchange` + `BroadcastHashJoin`	Small side copied to all executors; big side not shuffled
`SortMergeJoin`	Both sides shuffled and sorted; normal for two big tables
`HashAggregate(partial_...)` then `Exchange` then `HashAggregate`	Map-side partial aggregation, then final aggregation
`InMemoryTableScan`	Reading from a cached DataFrame
`AdaptiveSparkPlan isFinalPlan=false`	AQE is on; the final plan is decided at runtime

Caching example:

```
df = spark.read.parquet("sales").filter("amount > 500").cache()
df.count()                        # first action fills the cache
df.groupBy("region").count().show()
df.explain()                      # shows InMemoryTableScan
df.unpersist()                    # free the memory when done
```

Quick recall

- Spark is a processing engine; it does not store data.
- One action = one job. Each shuffle splits a job into another stage. One task per partition per stage.
- Driver death kills the app; executor death only reruns tasks.
- Lineage replaces replication for intermediate data; `checkpoint()` cuts long lineage.
- Narrow: `map`, `filter`. Wide (shuffle): `groupByKey`, `reduceByKey`, `join`, `distinct`, `repartition`.
- Prefer `reduceByKey` over `groupByKey` for aggregations.
- DataFrames and SQL use Catalyst and Tungsten; RDD lambdas and Python UDFs are black boxes.
- Broadcast threshold default: 10 MB (`spark.sql.autoBroadcastJoinThreshold`).
- Default shuffle partitions for DataFrames: 200 (`spark.sql.shuffle.partitions`).
- Skew: one slow task in a stage. Fix with salting, AQE skew join, or broadcast.
- Cache only data used more than once; `unpersist()` when done.
- `spark.ml` (DataFrames) is the current ML API; `spark.mllib` (RDDs) is maintenance only.

Most-Asked CAT and Exam Questions (Days 1 to 4)

Forty questions that come up again and again in CATs and exams on foundations, Hadoop, Hive, Pig and Spark, with short answers you can write in an exam.

---

1. Foundations

Q1. Define Big Data using the 5 V's. Give an example of each.

Big Data is data whose size, speed or variety is too much for one machine or a traditional database to store and analyse in acceptable time and cost.

V	Meaning	Example (mobile money operator)
Volume	Amount of data	Billions of transactions per year
Velocity	Speed it arrives and must be handled	Thousands of payments per second, fraud checks in real time
Variety	Different formats	Transaction tables, SMS text, app logs (JSON), call audio
Veracity	Quality and trust	Duplicate records, missing agent IDs, wrong timestamps
Value	Benefit after analysis	Detecting fraud, predicting churn

Q2. Name the four types of analytics and give one example of each.

Type	Question it answers	Example
Descriptive	What happened?	Monthly sales report per region
Diagnostic	Why did it happen?	Drill-down shows sales fell because one store closed
Predictive	What will happen?	Forecast next month's demand
Prescriptive	What should we do?	Decide how much stock to send to each store

Each level is harder to build and gives more business value than the one before.

Q3. Compare vertical and horizontal scaling. Which one do Big Data systems use and why?

- **Vertical (scale up):** buy a bigger machine (more CPU, RAM, disk). Simple, no code change, but there is a hardware ceiling and cost rises fast.
- **Horizontal (scale out):** add more ordinary machines to a cluster. No hard ceiling and cheaper per unit, but you must deal with partitioning, network failures and consistency.

Big Data platforms (HDFS, Spark, Cassandra) scale out, because data grows past what any single machine can hold and commodity servers are cheaper.

Q4. State the CAP theorem. Where do HBase, Cassandra and MongoDB sit, and why can a distributed system not simply choose CA?

CAP: during a network partition a distributed data store cannot give both Consistency (every read sees the latest write) and Availability (every request gets a response). Partition tolerance is not optional, because networks do fail; so the real choice during a partition is C or A.

System	Usual placement	Why
HBase	CP	Each region is served by one RegionServer; if it is unreachable, that data is unavailable until recovery
MongoDB	CP (default, majority write concern)	Writes go to the primary; during an election writes pause
Cassandra	AP	Any replica accepts writes; replicas reconcile later. Consistency is tunable per query (QUORUM, ALL)

"CA" only exists for a single-node database, where there is no network partition to tolerate.

Q5. Compare ACID and BASE. Give a use case for each.

	ACID	BASE
Stands for	Atomicity, Consistency, Isolation, Durability	Basically Available, Soft state, Eventual consistency
Priority	Correctness of every transaction	Availability and scale
Typical systems	PostgreSQL, MySQL, Oracle	Cassandra, DynamoDB, many NoSQL stores
Use case	Bank transfer: money must never appear or vanish	Social media likes counter: being a few seconds stale is fine

Q6. Compare the Lambda and Kappa architectures.

- **Lambda:** two paths. A batch layer recomputes accurate results from all historical data; a speed layer processes recent data in real time; a serving layer merges both. Accurate and fresh, but you write and maintain the same logic twice.
- **Kappa:** one streaming path for everything. To reprocess history you replay the stored stream (for example from Kafka). One code base, but it needs a durable, replayable log and mature streaming tools.

Choose Lambda when batch accuracy is critical and the team already has batch jobs. Choose Kappa when most logic is event-driven and the log can hold enough history.

---

2. HDFS and YARN

Q7. Distinguish the NameNode, the Secondary NameNode and the Standby NameNode.

	Role	Is it a hot backup?
NameNode (active)	Holds the namespace and block map in memory; answers all metadata requests	It is the master
Secondary NameNode	Periodically merges the edit log into the fsimage (checkpointing) so restarts are fast	**No.** It cannot take over
Standby NameNode (HA)	Keeps an up-to-date copy of metadata via JournalNodes; ZooKeeper failover promotes it if the active fails	Yes

The Secondary NameNode is a common trick question: despite the name, it is a checkpoint helper, not a failover node.

Q8. Why is the HDFS block size large (128 MB) compared with a normal file system (4 KB)?

- **Less metadata:** the NameNode keeps every block in RAM. Bigger blocks mean fewer blocks per file and less memory.
- **Disk efficiency:** large sequential reads spend most of their time transferring data, not seeking.
- **Sensible task size:** one map task usually processes one block, so 128 MB gives tasks that run long enough to be worth scheduling.

Too large a block size reduces parallelism (fewer tasks); too small overloads the NameNode.

Q9. A 1 GB file is stored in HDFS with 128 MB blocks and replication factor 3. How many blocks, block replicas and raw storage? What changes for a 300 MB file?

- 1 GB = 1024 MB. 1024 / 128 = **8 blocks**.
- Each block has 3 copies: 8 × 3 = **24 block replicas**.
- Raw storage: about 1 GB × 3 = **3 GB**.

For a 300 MB file: ceil(300 / 128) = 3 blocks (128, 128, 44 MB), 9 replicas, about 900 MB raw. The last block only uses 44 MB on disk; HDFS does not pad it to 128 MB.

Q10. What is the small files problem and how do you fix it?

Every file, directory and block is an object in NameNode memory (roughly 150 bytes each). Millions of tiny files fill the NameNode heap even when the total data is small. They also create one map task (or Spark task) per file, so jobs spend more time starting tasks than processing data.
Fixes:

- merge small files before or after loading (a compaction job);
- write with fewer output partitions (`coalesce` in Spark);
- store data in container formats such as ORC, Parquet, Avro or SequenceFiles;
- use Hadoop Archives (HAR) for cold data;
- avoid over-partitioning Hive tables.

Q11. What happens when a DataNode dies?

1. The DataNode stops sending heartbeats (every 3 seconds by default).
2. After about 10 minutes without heartbeats, the NameNode marks it dead.
3. The NameNode finds every block that now has fewer than 3 live replicas.
4. It tells other DataNodes to copy those blocks from surviving replicas until the replication factor is restored.

Clients that were reading from the dead node simply switch to another replica. No data is lost unless all replicas of a block fail at the same time.

Q12. When you run Spark on YARN, what is the difference between client mode and cluster mode?

	Client mode	Cluster mode
Where the driver runs	On the machine where you ran `spark-submit`	Inside a YARN container (in the ApplicationMaster)
Output of `print` / `show`	Your terminal	YARN container logs
If your laptop disconnects	Job dies	Job keeps running
Use for	Interactive work, notebooks, debugging	Scheduled production jobs

```
spark-submit --master yarn --deploy-mode cluster job.py
```

---

3. MapReduce

Q13. Distinguish a combiner from a partitioner.

	Combiner	Partitioner
Purpose	Shrink map output before the shuffle	Decide which reducer receives each key
Runs	On the mapper side, on one mapper's output	On the mapper side, for every output record
Default	None (optional)	Hash partitioner: `hash(key) mod R`
Guarantee	May run zero, one or many times	Always runs when there are reducers
Affects	Network traffic	Load balance between reducers and which keys meet

A combiner is only safe for associative and commutative operations (sum, count, max). A custom partitioner is used to send related keys together or to spread a hot key.

Q14. What is speculative execution?

When one task runs much slower than others in the same phase (a straggler), the framework launches a duplicate copy of it on another node. Whichever copy finishes first is used and the other is killed.
It helps when the slowness comes from a bad node (failing disk, busy machine). It does not help with data skew, because the duplicate gets the same oversized input. It uses extra resources, so it is often turned off on busy clusters. In Hadoop it is on by default; in Spark it is off by default (`spark.speculation=false`).

Q15. Trace Word Count on the two input lines "big data is big" and "data is fast".

Map output:

- line 1: (big,1) (data,1) (is,1) (big,1)
- line 2: (data,1) (is,1) (fast,1)

After a combiner on mapper 1: (big,2) (data,1) (is,1).
Shuffle and sort (grouped by key):

- big → [2]
- data → [1, 1]
- fast → [1]
- is → [1, 1]

Reduce (sum): (big,2) (data,2) (fast,1) (is,2).

Q16. Why is it wrong to use "average" as a combiner, and how do you fix it?

The average of averages is not the overall average when groups have different sizes. Example: mapper A sees 10 and 20 (avg 15), mapper B sees 60 (avg 60). Average of averages = 37.5, but the true average of 10, 20, 60 is 30.
Fix: emit (sum, count) pairs. The combiner adds sums and counts; the reducer adds them again and divides once at the end: (30, 2) + (60, 1) = (90, 3), so 90 / 3 = 30.

Q17. Why is MapReduce slow for iterative algorithms such as k-means or PageRank?

Each iteration is a separate MapReduce job. Every job reads its input from HDFS, writes map output to local disk, shuffles, and writes the final output back to HDFS with 3 replicas. The next iteration reads it all again, and each job also pays start-up and scheduling cost. With 50 iterations, most of the time goes to disk I/O and job overhead, not computation. Spark fixes this by caching the dataset in memory between iterations.

---

4. Hive and Pig

Q18. A team lands JSON event logs in HDFS every hour. How can Hive query them without first converting them into tables?

Hive uses **schema-on-read**. You create an external table that describes the columns and points at the folder, with a JSON SerDe to parse each line. The files are not changed or validated when they land; the schema is applied only when a query reads them.

```
CREATE EXTERNAL TABLE events (user_id STRING, action STRING, ts TIMESTAMP)
ROW FORMAT SERDE 'org.apache.hive.hcatalog.data.JsonSerDe'
LOCATION '/data/raw/events';
```

Trade-off: loading is instant and flexible, but bad records only show up as NULLs or errors at query time.

Q19. Compare managed and external tables. What happens on DROP TABLE?

	Managed (internal)	External
Who owns the files	Hive	Someone else (Hive only reads them)
Location	Hive warehouse directory	Any path given with `LOCATION`
`DROP TABLE`	Deletes metadata **and** data files	Deletes metadata only; files stay
Use for	Tables Hive creates and controls	Shared raw data used by other tools (Spark, Pig)

Q20. Compare ORC, Parquet and Avro.

	ORC	Parquet	Avro
Layout	Columnar	Columnar	Row-based
Best for	Hive analytics, Hive ACID tables	Analytics across many engines (Spark, Trino, Impala)	Whole-record pipelines, Kafka, ETL
Reads only needed columns	Yes	Yes	No
Predicate pushdown with min/max stats	Yes	Yes	No
Schema evolution	Good	Good	Very good

Rule of thumb: Avro for moving records, Parquet or ORC for querying them.

Q21. Write a Hive table partitioned by date, load it with dynamic partitioning, and write a query that uses partition pruning.

```
CREATE TABLE sales (sale_id BIGINT, region STRING, amount DOUBLE)
PARTITIONED BY (dt STRING)
STORED AS ORC;

SET hive.exec.dynamic.partition.mode=nonstrict;
INSERT OVERWRITE TABLE sales PARTITION (dt)
SELECT sale_id, region, amount, dt FROM sales_raw;   -- partition column last

SELECT region, SUM(amount)
FROM sales
WHERE dt = '2024-03-15'          -- reads one folder only
GROUP BY region;
```

Pruning works because the filter is directly on the partition column. Wrapping it in a function (for example `WHERE year(dt) = 2024` on a string column) can stop pruning. Avoid partitioning on high-cardinality columns such as `sale_id`: it creates millions of tiny folders.

Q22. Write Word Count in Pig Latin and explain each line.

```
lines  = LOAD 'input' AS (line:chararray);                    -- one tuple per line
words  = FOREACH lines GENERATE FLATTEN(TOKENIZE(line)) AS word; -- split into words, one per
row
groups = GROUP words BY word;                                  -- bag of rows per word
counts = FOREACH groups GENERATE group AS word, COUNT(words) AS n;
STORE counts INTO 'output';                                    -- triggers execution
```

`TOKENIZE` returns a bag of words and `FLATTEN` turns that bag into separate rows. Nothing runs until `STORE` (or `DUMP`), similar to Spark actions.

---

5. Spark core

Q23. What is a DAG in Spark and how does Spark turn it into stages and tasks?

A DAG (directed acyclic graph) is the chain of transformations from the source data to the action, with no loops. When an action runs, the DAG scheduler walks the graph backwards from the result and cuts it at every wide dependency (shuffle). Each piece between cuts is a stage. Narrow steps inside a stage are pipelined, so data flows through `map` then `filter` in one pass. Each stage runs one task per partition.

Q24. Compare cache(), persist() and checkpoint().

	`cache()`	`persist(level)`	`checkpoint()`
Stores	Default level (RDD: memory only; DataFrame: memory and disk)	Level you choose (`MEMORY_ONLY`, `MEMORY_AND_DISK`, `DISK_ONLY`, `_2` replicated)	Reliable storage such as HDFS
Keeps lineage	Yes	Yes	**No**, lineage is cut
Survives executor loss	Recomputed from lineage	Recomputed (unless replicated level)	Yes, read back from storage
Use for	Data reused a few times	Fine control over memory vs disk	Long lineage in iterative jobs

```
from pyspark import StorageLevel
df.persist(StorageLevel.MEMORY_AND_DISK)
sc.setCheckpointDir("hdfs:///tmp/ckpt")
rdd.checkpoint()     # happens at the next action
```

Q25. Compare repartition() and coalesce().

- `repartition(n)` does a full shuffle and gives evenly sized partitions. It can increase or decrease the count, and can partition by a column (`repartition("region")`).
- `coalesce(n)` only decreases the count by merging existing partitions on the same executor, with no full shuffle. It is cheap but partitions can end up uneven.

Use `repartition` to raise parallelism or prepare for a join on a key. Use `coalesce` before writing output to avoid many small files.

Q26. What are broadcast variables and accumulators?

- **Broadcast variable:** a read-only value (for example a lookup dictionary) sent once to each executor instead of once with every task.
- **Accumulator:** a write-only counter that tasks add to and only the driver reads, for example to count bad records.

```
rates = sc.broadcast({"RWF": 1.0, "USD": 1300.0})
bad = sc.accumulator(0)

def convert(row):
    if row[1] not in rates.value:
        bad.add(1)
        return None
    return row[0] * rates.value[row[1]]

total = rdd.map(convert).filter(lambda x: x is not None).sum()
print(bad.value)
```

Caution: updates made inside transformations can be counted twice if a task is retried. Only updates inside actions (`foreach`) are guaranteed to be applied once.

Q27. Count the jobs and stages (exercise 1).

```
rdd = sc.textFile("data.txt")
counts = rdd.flatMap(lambda l: l.split()).map(lambda w: (w, 1)).reduceByKey(lambda a, b: a +
        b)
counts.count()
counts.collect()
```

- Jobs: **2** (two actions: `count` and `collect`).
- Stages per job: 2 (`reduceByKey` is the only shuffle). Stage 1: `textFile`, `flatMap`, `map`. Stage 2: the reduce side.
- In the second job the first stage shows as **skipped** in the Spark UI, because its shuffle output from the first job is reused. So 4 stages are planned, 3 actually run.

Q28. Count the jobs, stages and tasks (exercise 2).

```
a = sc.parallelize(range(100), 4).map(lambda x: (x % 10, x))
b = a.reduceByKey(lambda x, y: x + y)
c = b.map(lambda kv: (kv[1] % 3, kv[0])).groupByKey()
c.collect()
```

- Jobs: **1** (one action).
- Stages: **3** (two shuffles: `reduceByKey` and `groupByKey`).
- Tasks: 4 per stage, so **12**. Shuffle outputs keep the parent's 4 partitions when `spark.default.parallelism` is not set and no partition count is passed.

Q29. Count the jobs and stages (exercise 3, with cache).

```
from operator import add
errors = sc.textFile("logs").filter(lambda l: "ERROR" in l).cache()
errors.count()                                                    # action 1
errors.filter(lambda l: "disk" in l).count()                      # action 2
errors.map(lambda l: (l.split()[0], 1)).reduceByKey(add).collect()  # action 3
```

- Jobs: **3**.
- Job 1: 1 stage. Reads the file, filters, and fills the cache.
- Job 2: 1 stage. Reads from the cache, not the file.
- Job 3: 2 stages (shuffle at `reduceByKey`). The first stage reads from the cache.
- Total: **4 stages**. Without `cache()` the log file would be read 3 times.

For DataFrames the count is less exact: `show()`, schema inference, broadcast joins and AQE can start extra small jobs. Use the Spark UI to check.

---

6. Spark SQL and optimization

Q30. Name the phases a Spark SQL query goes through before it runs.

1. **Parse:** SQL text or DataFrame calls become an unresolved logical plan.
2. **Analyze:** table and column names are checked against the catalog and schema.
3. **Optimize (logical):** Catalyst rules rewrite the plan (push filters down, prune columns, fold constants).

4. **Physical planning:** choose concrete operators, for example broadcast join or sort-merge join, using size estimates.
5. **Code generation:** Tungsten compiles stages into Java bytecode, then tasks run on executors.

SQL and the DataFrame API go through exactly the same steps, so they perform the same.

Q31. What is Adaptive Query Execution (AQE)?

AQE re-optimizes a query while it runs, using real statistics from finished shuffle stages instead of guesses. It is on by default from Spark 3.2. It does three main things:

- merges many small shuffle partitions into fewer, larger ones;
- switches a sort-merge join to a broadcast join when one side turns out to be small;
- splits skewed partitions in joins into smaller pieces.

In `explain()` you see `AdaptiveSparkPlan`.

Q32. Why are Python UDFs slow, and what should you use instead?

A plain Python UDF sends each row from the JVM to a Python worker process and back. That serialization is costly, and the function is a black box to Catalyst, so no pushdown or code generation happens through it.

Better options, in order:

1. built-in functions (`pyspark.sql.functions`: `when`, `regexp_extract`, `date_add`, etc.);
2. `pandas_udf` (vectorized, uses Apache Arrow, processes batches instead of single rows);
3. a Scala or Java UDF registered in Spark, if the logic is complex.

Q33. What is the difference between predicate pushdown and partition pruning?

- **Partition pruning:** skips entire folders because the filter is on the partition column (`dt = '2024-03-15'` reads only `dt=2024-03-15/`). Shown as `PartitionFilters` in `explain()`.
- **Predicate pushdown:** sends a filter on a normal column into the file reader. Parquet and ORC use min/max statistics per row group to skip blocks that cannot match. Shown as `PushedFilters`.

Both reduce data read. Pruning works at folder level using metadata; pushdown works inside files.

Q34. What does spark.sql.shuffle.partitions control, and how should you set it?

It sets how many partitions (and so tasks) a DataFrame shuffle creates for joins and aggregations. The default is 200.

- Too high for small data: hundreds of tiny tasks and tiny output files; overhead dominates.
- Too low for big data: huge partitions that spill to disk or run out of memory.

A common target is 100 to 200 MB of shuffle data per partition, and at least 2 to 3 times the total number of cores. With AQE on, set it on the high side and let AQE merge small partitions.

---

7. MLlib and GraphX

Q35. Explain Transformer, Estimator and Pipeline with an example.

- **Transformer:** has `transform(df)` and returns a new DataFrame, for example `VectorAssembler` or a fitted model.
- **Estimator:** has `fit(df)` and returns a Transformer, for example `StringIndexer` or `LogisticRegression`.
- **Pipeline:** a list of stages. `fit` runs them in order and returns a `PipelineModel` that applies the same steps to new data.

```python
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, VectorAssembler
from pyspark.ml.classification import LogisticRegression

idx = StringIndexer(inputCol="category", outputCol="category_idx")
vec = VectorAssembler(inputCols=["category_idx", "age_days"], outputCol="features")
lr = LogisticRegression(featuresCol="features", labelCol="label")

model = Pipeline(stages=[idx, vec, lr]).fit(train)
predictions = model.transform(test)
```

Pipelines prevent a common bug: training and prediction data being prepared differently.

Q36. A fraud model shows 99% accuracy but catches almost no fraud. Why, and which metrics should you report?

Because the classes are imbalanced. If only 1% of transactions are fraud, a model that always says "not fraud" is 99% accurate and useless.

Report instead:

- **Recall:** of all real fraud, how much did we catch?
- Precision: of all flagged cases, how many were real fraud?
- F1 score (balance of the two) and area under the precision-recall curve.
- A confusion matrix.

Fixes include class weights, resampling, and choosing the decision threshold based on cost.

Q37. Explain PageRank and the Pregel (superstep) model in simple terms.

PageRank scores a vertex by how many important vertices point to it. Every vertex starts with the same score. In each round it sends its score, split evenly, along its outgoing edges; each vertex adds up what it receives, applies a damping factor (usually 0.85), and repeats until scores stop changing.

Pregel runs graph algorithms this way: in each **superstep** every vertex reads messages from the last step, updates its value, and sends messages to neighbours. The algorithm stops when no messages remain or after a maximum number of steps. Each superstep is a distributed round with a barrier, so the number of iterations drives the runtime.

---

8. Choosing the right tool (scenario questions)

Q38. For each scenario, pick MapReduce, Hive, Pig or Spark and justify.

Scenario	Pick	Reason
Analysts need a monthly report over 5 TB of sales in a data lake	Hive (or Spark SQL)	SQL-natural, shared catalog, batch latency is fine
Train a churn model with 30 passes over 2 TB	Spark (MLlib)	Iterative; caching avoids rereading disk each pass
Legacy ETL already written as MapReduce, runs fine nightly	Keep MapReduce	Works; rewriting costs more than it saves
Multi-step cleaning of nested, messy logs by an engineering team	Pig or Spark DataFrames	Step-by-step dataflow; Spark is the modern choice
Rank users in a 2 billion edge social graph	Spark GraphX or GraphFrames	Built-in iterative graph algorithms
Sub-second dashboard over 10 GB	None of these	Use DuckDB, ClickHouse, Druid or precomputed tables

Q39. Design a small batch analytics pipeline for a court case system (sources to dashboard), naming a tool for each layer.

1. **Sources:** the case management database (PostgreSQL), hearing logs, uploaded documents.
2. **Ingestion:** a nightly export of changed rows (Sqoop or Spark JDBC read); logs shipped as files.
3. **Storage:** raw zone in HDFS or object storage as received; curated zone as Parquet partitioned by date.
4. **Processing:** Spark jobs clean, join cases and hearings, and compute backlog per court and category.
5. **Analytics:** Spark SQL or Hive tables for analysts; MLlib to predict which cases risk long delays.
6. **Consumption:** small aggregate tables feed a dashboard (Superset, Power BI).
7. **Governance and operations:** Airflow schedules and retries the jobs; access control on sensitive case data; data quality checks; lineage recorded in a catalog.

Consistency choice: the source case database stays ACID (CP); the analytics copy can be a day old, which is acceptable for reporting.

Q40. Your whole dataset is 2 GB and fits on a laptop. Should you use Hadoop or Spark?

Usually no. At this size, pandas, Polars or DuckDB on one machine are faster and simpler: no cluster to run, no shuffle, no job overhead. Distributed tools pay off when data does not fit on one machine, when processing on one machine takes too long, or when the data is growing fast toward that point. In an exam, state the size threshold, the cost of running a cluster, and the growth expectation as your reasons.

Quick recall

- 5 V's: Volume, Velocity, Variety, Veracity, Value.
- CAP: during a partition choose C or A. HBase and MongoDB lean CP; Cassandra leans AP.
- Secondary NameNode checkpoints metadata; Standby NameNode is the real failover.
- HDFS default: 128 MB blocks, replication 3, heartbeat every 3 s.
- Blocks = ceil(file size / block size); raw storage is about file size × replication.
- Combiner reduces shuffle volume; partitioner decides the reducer (`hash(key) mod R`).
- Never average averages: carry (sum, count).
- Dropping an external Hive table keeps the files; dropping a managed table deletes them.
- Partition pruning skips folders; predicate pushdown skips blocks inside files.
- One action = one job; each shuffle adds a stage; one task per partition.
- `coalesce` to reduce files cheaply; `repartition` to rebalance with a shuffle.
- Accuracy misleads on imbalanced data; report recall, precision and F1.